Automated Current-Mode Bandgap Reference Design Flow

Overview

This project implements a **fully automated design and optimization flow for a CMOS current-mode bandgap reference**. The flow combines **analytical bandgap theory**, **gm/Id-based transistor synthesis**, and **simulation-in-the-loop optimization** using **Python** and **Cadence Spectre**.

The objective is to minimize manual intervention while achieving a temperature-stable, low-noise reference voltage that satisfies power, accuracy, and noise constraints in scaled CMOS technologies.

Key Features

- Analytical pre-sizing of resistors and currents using bandgap equations
- gm/Id-based automated transistor sizing from Spectre lookup tables
- Three-stage DC temperature optimization:
  - Smoothness enforcement
  - Reference voltage centering
  - Turning-point temperature control
- Noise-aware optimization targeting dominant contributors
- Fully script-driven interaction with Cadence Spectre
- Reproducible and extensible automation framework

Design Specifications (Typical)

- Supply Voltage: 1.2 V
- Target Reference Voltage: 0.5 V
- Temperature Range: $-40°C$ to $125°C$
- Power Consumption: $\leq$ specification limit
- Output Noise and 1/f Corner: User-defined

All specifications are defined in a single Python configuration file.

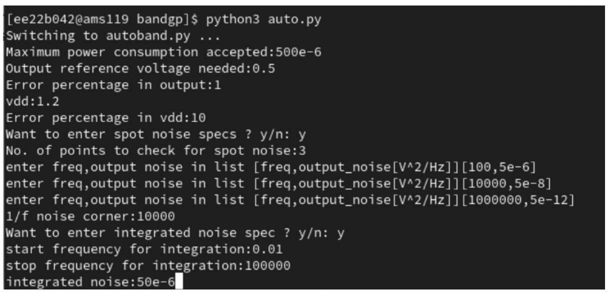

Figure 1: Input Specifications

Project Structure

```
project_root/

          auto.py                    # Top-level entry script
          autoband.py                # Main DC optimization flow

          calculations.py            # Analytical pre-sizing
          spec.py                    # User specifications

          netlists/
                band_gap.scs
                diode.scs
                nmoschar.scs
                pmoschar.scs

          utils/
                psf_parser.py
                noise_utils.py
                optimization_utils.py

          band_gap/psf/
                dc.dc/
                noise.noise/
                logs/

          README.md
```

Design Flow Description

1. Analytical Pre-Sizing

- Diode temperature sweep to extract $V_{BE}(T)$
- CTAT slope extracted from simulation data
- PTAT contribution derived using ΔV_{BE}
- Resistor ratios (R_1, R_2, R_3) and branch currents computed analytically

2. gm/Id-Based Device Synthesis

- NMOS and PMOS devices characterized using Spectre
- gm/Id lookup tables generated
- Target gm/Id mapped to current per finger
- Required finger counts and channel lengths synthesized automatically

3. DC Temperature Optimization

A three-stage simulation-driven optimization loop is used:

1. **Smoothness Enforcement**: Ensures a single, well-behaved $V_{out}(T)$ curve
2. **Reference Centering**: Aligns V_{out} at room temperature to the target value

3. **Turning-Point Control**: Constrains the turning-point temperature to a specified window

4. Noise and Flicker Optimization

- Spectre noise summaries parsed automatically
- Dominant noise contributors identified
- Selective device resizing:
  - Thermal noise → channel length increase
  - Flicker noise → length and width increase
- DC characteristics preserved during noise refinement

How to Run the Project

Prerequisites

- Python 3.0 or later with NumPy installed
- Cadence Spectre accessible from terminal
- Valid CMOS PDK (TSMC 65 GP) setup

Running the Automation

```
python3 auto.py
```

This command launches the default automation flow. No command-line flags are required.

Configuration

All design targets and constraints are defined in:

```
spec.py
```

Outputs

- Optimized transistor dimensions and resistor values
- V_{out} vs temperature characteristics
- Integrated noise and noise spectrum
- Flicker noise corner frequency
- Log files for each optimization run

Automation Philosophy

This framework is **knowledge-driven**, not black-box optimization.

- Analytical equations guide initial sizing
- gm/Id preserves analog design intuition
- Simulation validates every update

OTHER USEFUL FILES:

- Here is the link to lGoogle document Containing calculations
- Also included the schematic files which were used to create the netlist files **(OPAMP, CORE, START, top-bandgap)** inside bandgp folder.

Future Extensions

- Startup circuit integration and PVT automation
- Mismatch and Monte Carlo analysis
- Post-layout parasitic-aware optimization
- ML-assisted optimization loops

Author

Jaiveer Kiran S K
Department of Electrical Engineering
Indian Institute of Technology Madras

License